Project 2 Report

Training Data Methodology

Started with the provided train using the given splits. As a baseline, I started with the full schema, but then tried candidate30 and eventually landed on candidate40 since it seemed like the full schema was too much for the model to focus on the relevant parts. Then there were two augmentations applied to the original data:

- Broad synthetic: added synthetic examples based on the existing ones but with more focus on some of the errors that were seen during some first runs. Focused on single-table examples, examples that require outputting empty column lists, and foreign key examples so there are more cases of join-linked tables.
- Hard negative: added examples that focused on naming the correct table but also referencing nearby tables to cover the fact that the model early on often chose incorrect tables but those that were semantically similar to the correct ones

Both augmentations were added off of visually seeing which cases seemed to fail the most during the early experiments. They helped have better coverage since the initial set of 301 seemed to not be enough, and helped train the model on selecting correct tables.
There were no additional diagnostic metrics defined, evaluation was mostly visual by looking through the per-question examples as well as seeing the precision and recall numbers to know what to focus on.

Final Pipeline Architecture

- Base model: `Qwen/Qwen2.5-1.5B-Instruct`
- Adapter: LoRA with `r=32`, `alpha=64`, dropout `0.05`, targeting `q_proj`, `k_proj`, `v_proj`, and `o_proj`
- Prompt template: “You are a schema linking model. Given a natural language question and a database schema, return only valid JSON mapping referenced table names to lists of referenced column names. Use only identifiers from the schema. If a table is referenced without specific columns, include it with an empty list.
  Question: {row['question']}
  Database: {db_id}
  Schema:{schema lines, either full schema or candidate40 schema}
  Return schema_links JSON only.
- Decoding: generate with 256 max tokens and extract the first JSON object
- Post-processing: keep only the tables and columns that exist in the target schema, and sort table names and column lists for consistent outputs

Experimentation Methodology

RapidFireAI was used to vary the knobs described below and run each experiment. Multi-config setups with RFGridSearch were attempted early on, but weren't used too extensively throughout the project. First of all, each run was already lengthy enough, and multiple configs would take reasonably too long given the time constraints of DSMLP (as well as the various login issues and lack of always available GPU). Second, unlike project 1, there were less knobs that felt like they required fine tuning to find a perfect value for. Rather, there was more focus on large changes like augmenting the data or changing the model. It was also useful to have the runs all be singular so that there were more chances to analyze the outputs and help construct the augmented dataset.

The following knobs were the main ones focused on:

- Schema prompt: full schema felt like it'd be too much, and the first change from the base model was changing it to candidate 30 which immediately had significant leaderboard improvements. Later on, candidate 40 seemed to do a bit better as well. This knob felt relevant since using the full schema would likely be too noisy, so it was a matter of using less of it while not limiting it so much that we'd exclude the correct ones.
- Data augmentation: see more details in training data methodology. This was done later on and had slight improvements on the leaderboard score by mostly improving some certain common fail cases
- Base model: 1.5B was just larger and did better, not surprising and made sense to change since it's still within the allowed limits
- LoRA capacity: More capacity seemed to give a slight boost in learning finer distinctions, changed because it felt like the r16 could be underfitting.
- Epochs: Given the amount of data was fairly small, it made sense to increase the epochs up to 3, which had a good result on the leaderboard. Didn't increase epochs any further partially because the training was already taking a while, partially because too many epochs could result in overfitting without actually learning anything useful.

Configs and leaderboard scores:

#	Base model	Schema prompt	Training data	LoRA	Epochs	Leaderboard
1	Qwen2.5-0.5B-Instruct	Full schema	Released train	r16, alpha32	1	0.3078
2	Qwen2.5-0.5B-Instruct	Candidate30	Released train	r16, alpha32	1	0.4229
3	Qwen2.5-0.5B-Instruct	Candidate30	Released train	r16, alpha32	3	0.5159
4	Qwen2.5-0.5B-Instruct	Candidate30	Released train	r32, alpha64	3	0.5295
5	Qwen2.5-0.5B-Instruct	Candidate40	Released train	r32, alpha64	3	0.5558
6	Qwen2.5-1.5B-Instruct	Candidate40	Released train	r32, alpha64	3	0.6373
7	Qwen2.5-1.5B-Instruct	Candidate40	Broad augmented	r32, alpha64	3	0.6555
8	Qwen2.5-1.5B-Instruct	Candidate40 + FK/PK	Broad augmented	r32, alpha64	3	0.6191
9	Qwen2.5-1.5B-Instruct	Candidate40	Hard-negative augmented	r32, alpha64	3	0.6572

Results and Analysis

See table in the previous section for all of the configs, with config 9 being the best final model chosen, though only a slight improvement over config 7. As a general note, most of these improvements seemed to affect mostly the table scores, and column metrics did not see much improvement despite more than doubling the leaderboard score from the baseline to the final.

What worked:

- Candidate instead of full schema had a strong effect since it removed irrelevant tables and reduced noise, had probably the single largest impact on leaderboard score of nearly 0.12
- Increasing the model from 0.5B to 1.5B was the second largest impact with nearly 0.08. This one wasn't surprising since obviously having 1 billion more parameters is probably better, but was a significant improvement nevertheless.
- LoRA rank was a slight improvement but overall didn't seem to make a large enough difference, though compared to the other changes it did have the largest impact on column metrics which otherwise barely moved.
- Both of the augmentations were good, improving almost 0.02 over just using the released train set, with hard-negative being slightly better, likely because it was more closely based on actual observable issues rather than the attempt to just add more broad data types, though the difference was almost negligible, so just adding broad data was also helpful.

What didn't work:

- FK/PK setup. I thought that adding explicit FK/PK information into the prompt could help the model figure out the relationships, but that surprisingly drastically dropped the leaderboard score. My main guess is that it didn't actually help the model learn, and might've just added more textual noise without fixing any of the issues (which hard-negative did help fix later on).

With more time I'd try more augmentation strategies. Hard-negative was the most effective in the end, but the way it was made was somewhat rough, and I wonder if being more specific with those would help more. I'd also focus more on data that resulted in column-related issues since those metrics stayed at much lower F1 values compared to table-level scores. Additionally, some knobs like candidate number and number of epochs still weren't tested to the point where I'm confident that those final values are near the best possible, they were just better than some earlier alternatives. I'd also see if there were other base models that might work better. Qwen1.5B is obviously better than 0.5B, but there are other model families to try. That's not something I wanted to focus on too much with the given time constraints since from my experience, just changing models usually doesn't yield large benefits, and is more of just a random search. But with more time that's definitely worth exploring.

AI Tools Statement

AI assistants were used to help with the following tasks:

- Adapting the provided demo notebook to work with our data instead of the sample data as a starting point for the project
- Formatting both of the data augmentation files
- Helping with util files/functions
- Putting together the final submission so that it runs exactly according to the guidelines
- Figuring out minor bug locations and fixes